

# AI Usage Report

Project: CampusCare — Student Support & Ticket Management System

Edumerge Solutions

## Quick Summary (Mandatory Form)

### AI TOOL USED

ChatGPT and Google DeepMind Antigravity

### WHAT I ASKED AI TO DO

- Help design a college student support system and write a complete Software Requirements Specification (SRS) document.
- Build the full working application using Node.js, Express, a real SQLite database, and a React frontend.
- Add a ticket deadline tracker (SLA engine), user roles (Student, Staff, Admin), private staff notes, and an automated test suite.

### PROMPT THAT WAS MOST USEFUL

"Read the SRS document and build a working full-stack web application with persistent database storage, real APIs, authentication, and role permissions. Then write an automated test suite to check all 20 test cases from the SRS."

### CODE GENERATED BY AI

- Database tables and connections ( `server/db.js` )
- Ticket deadline and overdue calculator ( `server/slaEngine.js` )
- Login and role security middleware ( `server/auth.js` )
- Sample demo data with 16 users and 32 tickets ( `server/seed.js` )
- Backend API routes for tickets, comments, staff, and dashboards
- React frontend pages, forms, and charts ( `src/` )
- Automated test script ( `tests/test_runner.js` )

### CODE I MODIFIED

- Updated the authentication code so users can download CSV reports directly by clicking a link in their browser.
- Changed terminal command syntax so it runs smoothly on Windows PowerShell.
- Cleaned up the CSV export format and updated the automated test checker.

### AI OUTPUT THAT WAS WRONG / HAD ISSUES

- Windows Command Error:** The AI generated a setup command using `&&` to run multiple commands together. On Windows PowerShell, this caused a syntax error.
- CSV Download Login Issue:** The CSV download route only looked for an authorization header. When clicking a normal "Download CSV" button in the browser, the browser does not send this header, so the download failed with an unauthorized error.
- Test Checker Mismatch:** The test script expected the CSV file header to start with `"Ticket Number"` (with quotes), but the backend generated `Ticket Number` (without quotes).

### HOW I IDENTIFIED THE PROBLEM

- The PowerShell error showed up immediately in the terminal when trying to install packages.
- Clicking the CSV download link showed a `401 Unauthorized` message.
- Running `npm test` showed a mismatch on the CSV test check.

### HOW I FIXED IT

- Replaced `&&` with `;` in PowerShell commands and used `concurrently` in `package.json` to start both the frontend and backend together.

- 2. Allowed the authentication middleware to accept a login token from the URL ( ?token=... ) so browser downloads work instantly.
- 3. Standardized the CSV headers and updated the test script to check the real output format.

## Detailed Project & AI Usage Breakdown

### 1. What is CampusCare?

In most colleges, when students have questions about fees, attendance, ID cards, or certificates, they have to send emails, make phone calls, or stand in line outside administrative offices. Requests often get delayed or lost.

**CampusCare** solves this by giving the college a single, organized website:

- **Students** can submit a ticket, select a category and priority, track its progress, and chat with staff.
- **Support Staff** can see tickets assigned to their department, answer students publicly, write private internal notes to other staff, and resolve tickets.
- **College Administrators** can see an overview dashboard, check how long tickets take to resolve, spot overdue tickets, and balance staff workloads.

### 2. How the Two AI Tools Were Used

| Tool        | What it was used for                                                                                                              |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------|
| ChatGPT     | Brainstorming the concept, planning the features, and writing the detailed requirements document ( <code>srs.txt</code> ).        |
| Antigravity | Writing the actual code, creating the database, building the frontend pages, fixing bugs, and creating the automated test runner. |

### 3. Step-by-Step AI Usage

#### Phase 1: Planning and Requirements (with ChatGPT)

- I asked ChatGPT to help plan a student ticket system for a college.
- Together, we defined:
  - The 3 roles: Student, Support Staff, Admin.
  - The 5 ticket stages: `Open` → `In Progress` → `Waiting for Student` → `Resolved` → `Closed` .
  - Realistic deadlines: Urgent tickets get solved in 4 hours; Low priority in 72 hours.
  - The test scenarios (T-001 through T-020).
- ChatGPT compiled all of this into a 1,500-line SRS document ( `srs.txt` ).

#### Phase 2: Building the Application (with Antigravity)

- I loaded the project into the Antigravity IDE and instructed it to read `srs.txt` and create the project.
- Antigravity set up:
  - An **Express backend** using **SQLite** ( `better-sqlite3` ) to save data permanently on disk.
  - A seed script that creates realistic college data (1 Admin, 5 Staff members from different departments, 10 Students, and 32 realistic tickets).
  - A clean, modern **React web interface** using simple, clear CSS.
  - Screens for Login, Admin Dashboard, Staff Queue, Student Portal, Ticket Details, Create Ticket, Staff Directory, Analytics, and Settings.

#### Phase 3: Testing and Validation (with Antigravity)

- I asked Antigravity to create test cases to check if our code matched the SRS.
- It created:
  - An automated test file ( `tests/test_runner.js` ) that runs with `npm test` .
  - A manual testing checklist ( `SRS_TEST_CASES.md` ) showing exactly what to click and what to expect on screen.
- We ran the test suite and confirmed that **all 20 test cases pass (100% success rate)**.

### 4. Important Problems Encountered and How They Were Solved

During development, we did not blindly accept AI output. We ran the code, tested it, found problems, and fixed them:

## 1. Windows PowerShell command chaining

**THE PROBLEM:** The AI generated a script using `npm install A && npm install B`. On Windows PowerShell, `&&` is not supported in older versions, and the command stopped.

**THE FIX:** Changed the separator to `;` and configured `concurrently` in `package.json` so one command starts both the backend and the frontend cleanly.

## 2. Downloading CSV files in the browser

**THE PROBLEM:** We wanted administrators to be able to download a spreadsheet (CSV) of all tickets. The backend required a secret login header. But when a user clicks a normal download link ( `<a href= "...">` ), the web browser does not attach custom headers, causing a `401 Unauthorized` error.

**THE FIX:** Updated the backend security code ( `server/auth.js` ) so it also accepts the login token in the web address ( `?token=...` ). Now, clicking the "Export CSV" button downloads the file immediately.

## 3. Protecting Private Staff Notes

**THE PROBLEM:** Staff members need to write internal notes to each other (e.g., checking bank records) that students should never see. We needed to ensure no student could see these notes even if they opened their browser developer tools.

**THE FIX:** Implemented a rule directly in the database query: if the logged-in user is a student, the server never sends internal notes over the network. Only public replies are returned.

## 4. Requiring a Resolution Summary

**THE PROBLEM:** The SRS requires that staff cannot mark a ticket as "Resolved" without explaining how it was solved.

**THE FIX:** Both the website modal and the backend API block the resolution if the summary is empty or too short.

## 5. How I Validated the Finished Application

1. **Automated Testing:** Ran `npm test`. The test runner automatically tests:

- Login for Student, Staff, and Admin.
- Creating a ticket and generating IDs like `CC-2026-00101`.
- Blocking students from seeing other students' tickets.
- Assigning tickets to staff.
- Hiding internal staff notes from students.
- Blocking resolutions without a summary.
- Reopening resolved tickets.
- Highlighting overdue tickets when a deadline passes.
- Searching and filtering.
- Exporting CSV files.

*Result: 20 out of 20 tests passed.*

2. **Manual Visual Testing:** Opened `http://localhost:3000` in the browser:

- Clicked the one-click demo login buttons to test as Admin, Staff, and Student.
- Checked that charts, badges, and colors display properly.
- Submitted a new ticket and watched it appear immediately on the dashboard.
- Verified that data remains intact even after refreshing the page or restarting the server.

## 6. Conclusion

AI tools significantly accelerated building CampusCare:

- **ChatGPT** acted like a product manager, helping plan the requirements and write the specifications.
- **Antigravity** acted like a pair-programming partner, writing the code, setting up the database, and building the user interface.

Through human review, terminal testing, and debugging, all AI-generated code was verified, corrected where needed, and proven to work against the assignment requirements.